Step-1 Create Project from the scratch

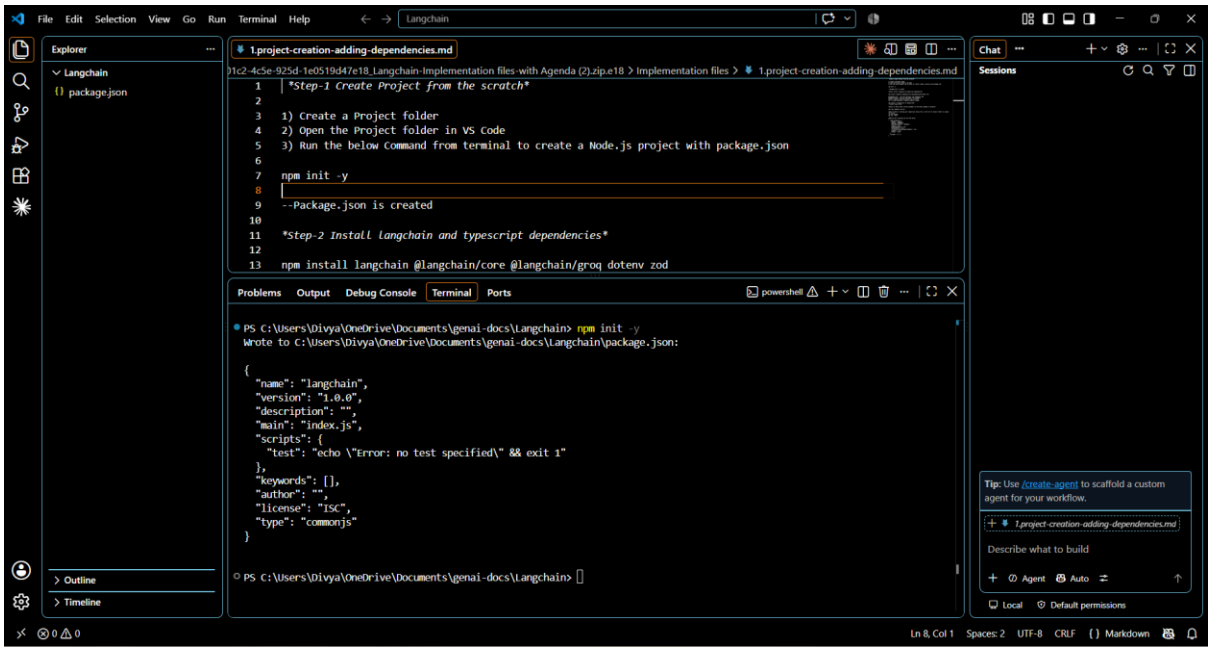
1) Create a Project folder

2) Open the Project folder in VS Code

3) Run the below Command from terminal to create a Node.js project with package.json

npm init -y

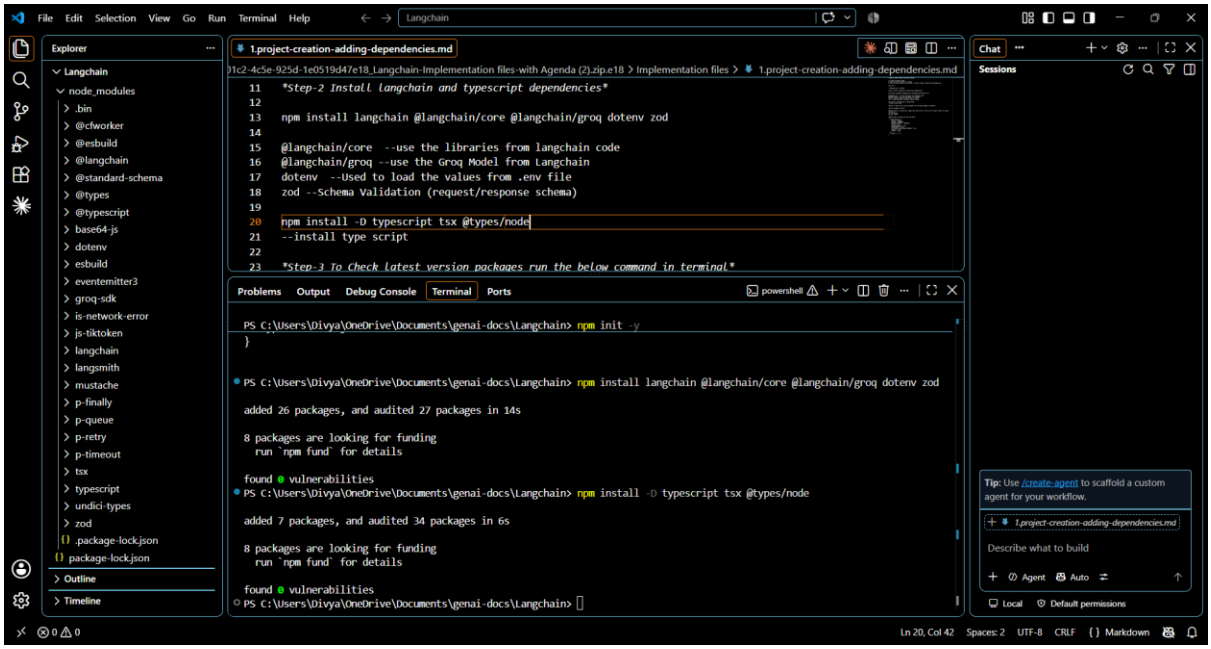
--Package.json is created



Step-2 Install langchain and typescript dependencies

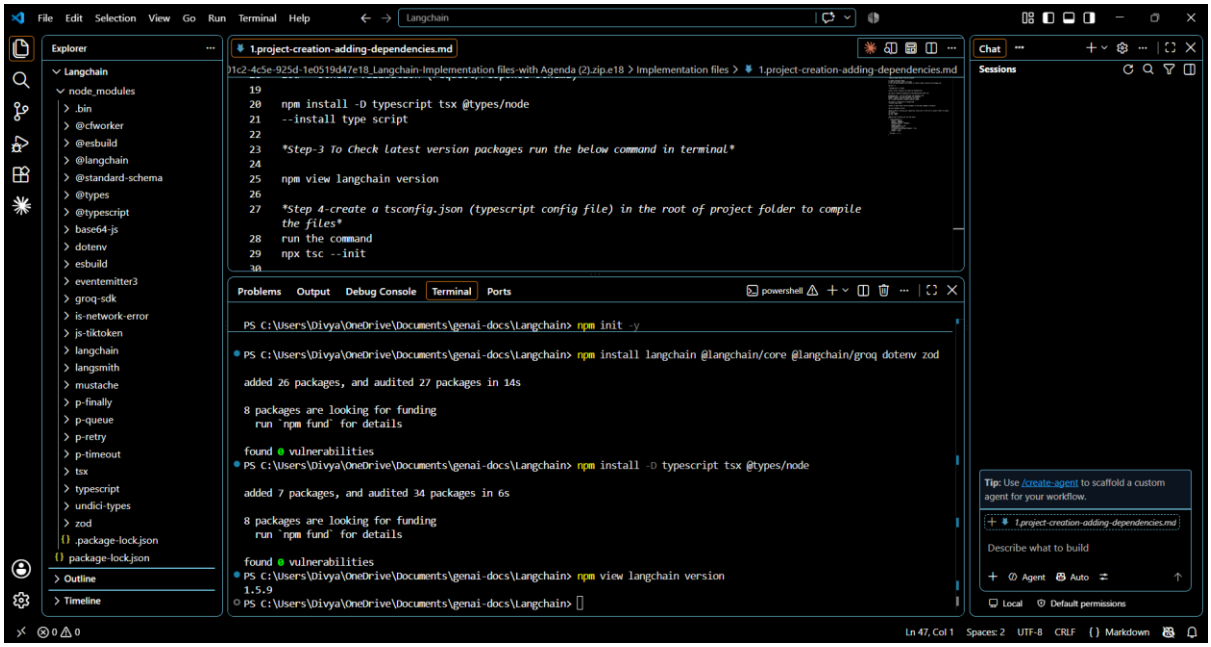
npm install langchain @langchain/core @langchain/groq dotenv zod

npm install -D typescript tsx @types/node



Step-3 To Check latest version packages run the below command in terminal

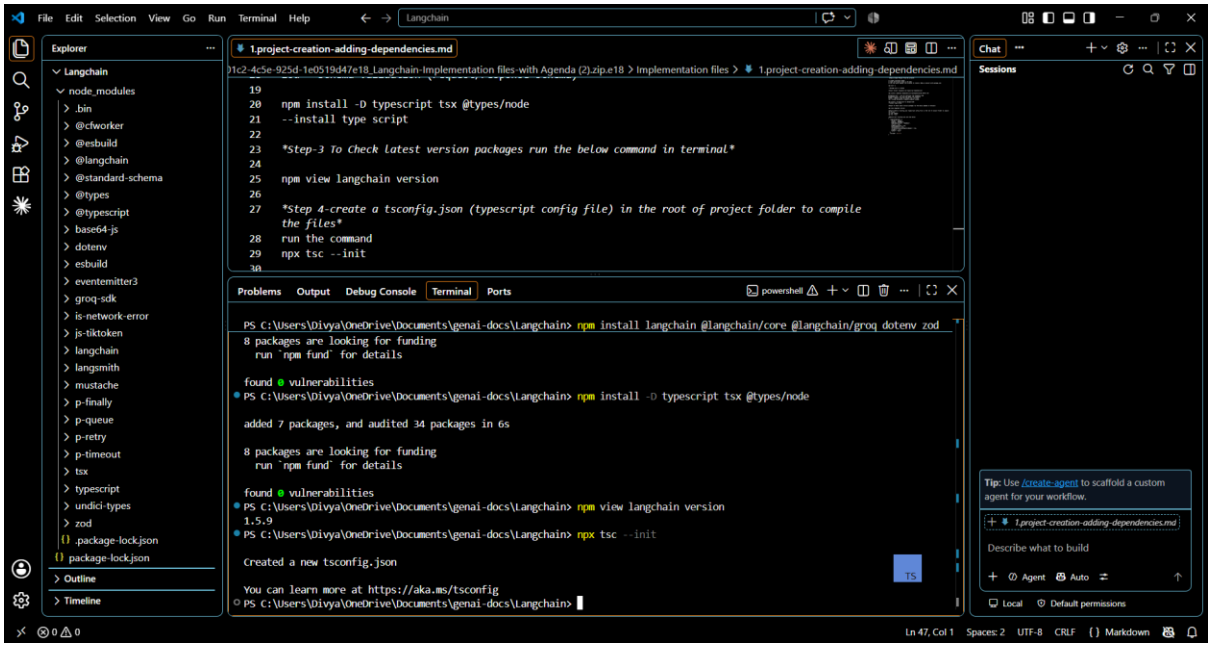
npm view langchain version



Step 4-create a tsconfig.json (typescript config file) in the root of project folder to compile the files

run the command

npx tsc --init

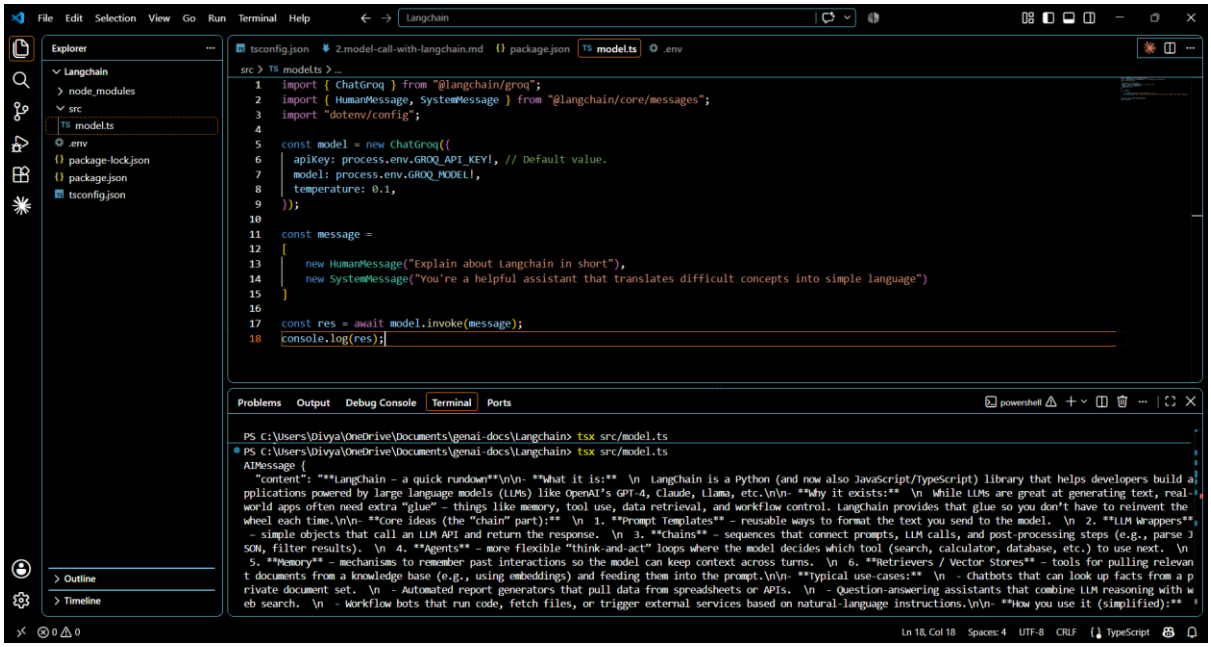


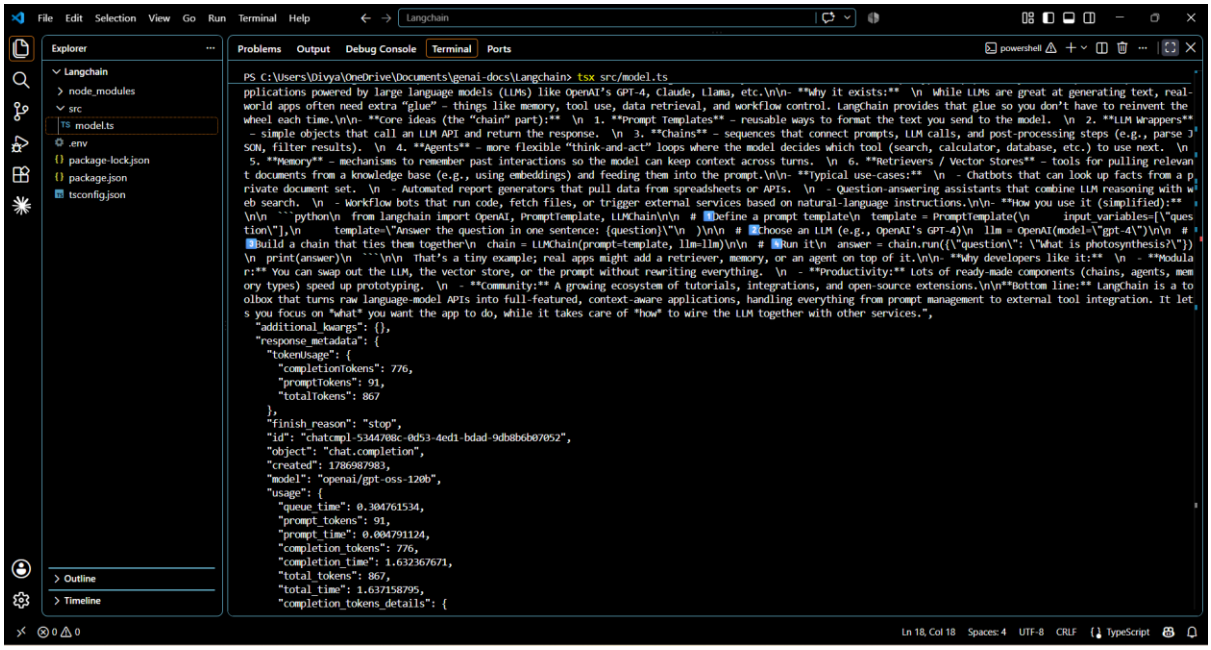
Replace your tsconfig.json with the below

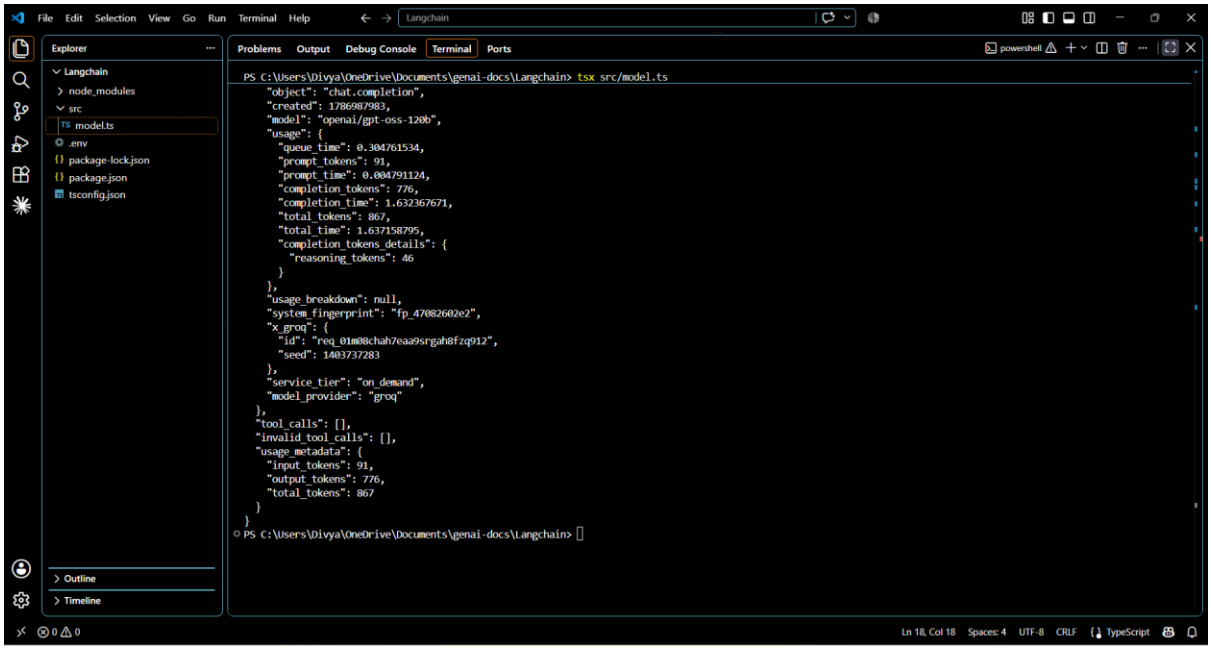
```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
```

```
  "moduleResolution": "NodeNext",
  "strict": true,
  "esModuleInterop": true,
  "skipLibCheck": true,
  "forceConsistentCasingInFileNames": true,
  "rootDir": "src",
  "outDir": "dist"
 },
 "include": ["src"]
}
```

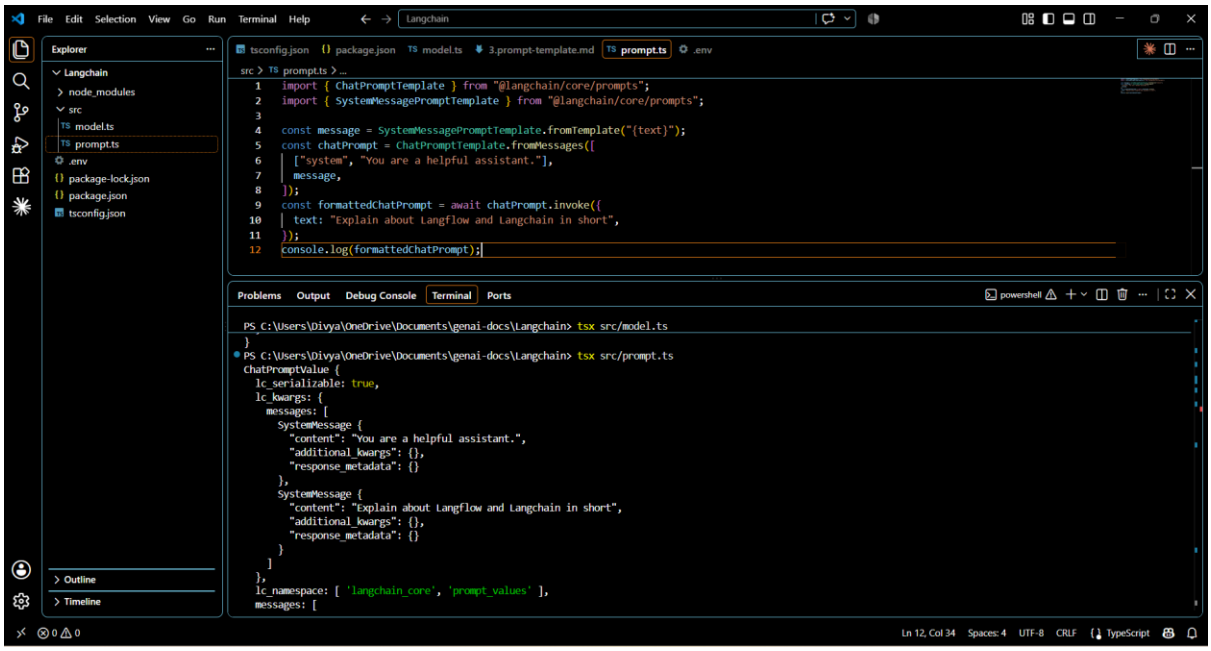
Model.ts

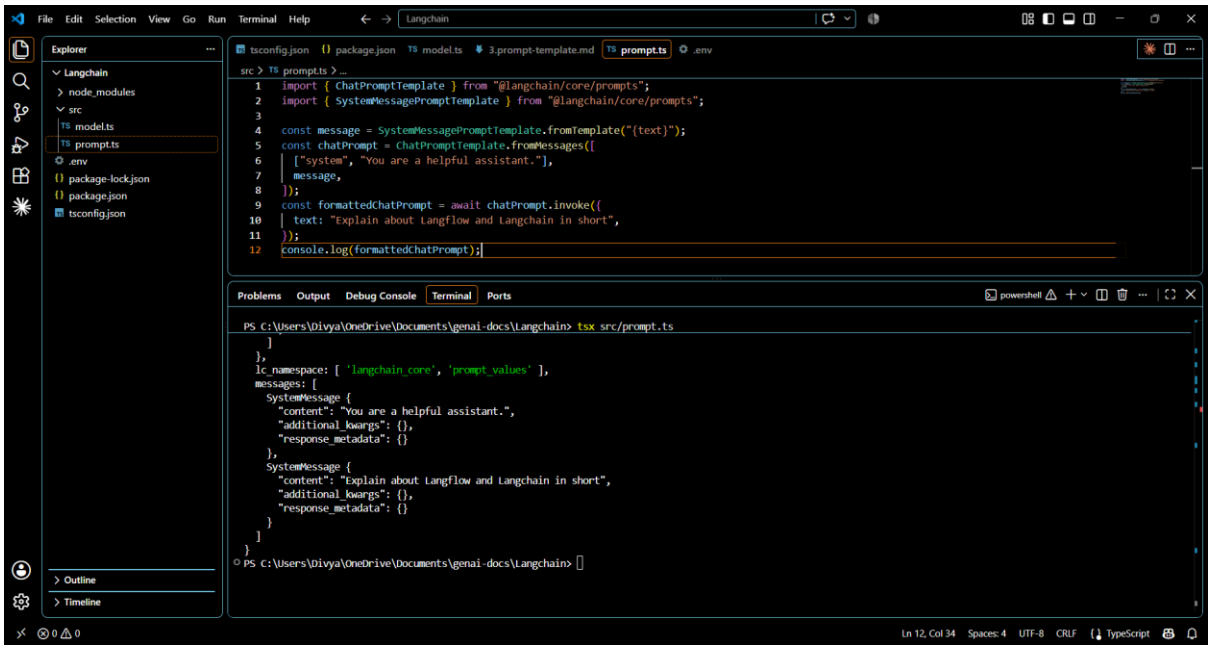



File Edit Selection View Go Run Terminal Help
Langchain
Explorer
Langchain
node_modules
src
model.ts
.env
package-lock.json
package.json
tsconfig.json
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/model.ts
pplications powered by large language models (LLMs) like OpenAI's GPT-4, Claude, Llama, etc.\n\n- **Why it exists:** \n While LLMs are great at generating text, real-world apps often need extra "glue" - things like memory, tool use, data retrieval, and workflow control. LangChain provides that glue so you don't have to reinvent the wheel each time.\n\n- **Core ideas (the "chain" part):** \n 1. **Prompt Templates** - reusable ways to format the text you send to the model. \n 2. **LLM Wrappers** - simple objects that call an LLM API and return the response. \n 3. **Chains** - sequences that connect prompts, LLM calls, and post-processing steps (e.g., parse JSON, filter results). \n 4. **Agents** - more flexible "think-and-act" loops where the model decides which tool (search, calculator, database, etc.) to use next. \n 5. **Memory** - mechanisms to remember past interactions so the model can keep context across turns. \n 6. **Retrievers / Vector Stores** - tools for pulling relevant documents from a knowledge base (e.g., using embeddings) and feeding them into the prompt.\n\n- **Typical use-cases:** \n - Chatbots that can look up facts from a private document set. \n - Automated report generators that pull data from spreadsheets or APIs. \n - Question-answering assistants that combine LLM reasoning with web search. \n - Workflow bots that run code, fetch files, or trigger external services based on natural-language instructions.\n\n- **How you use it (simplified):** \n\n ```python\n from langchain import OpenAI, PromptTemplate, LLMChain\n\n # 1. Define a prompt template\n template = PromptTemplate(\n input_variables=[\"question\"],\n template=\"Answer the question in one sentence: {question}\"\n)\n\n # 2. Choose an LLM (e.g., OpenAI's GPT-4)\n llm = OpenAI(model=\"gpt-4\")\n\n # 3. Build a chain that ties them together\n chain = LLMChain(prompt=template, llm=llm)\n\n # 4. Run it\n answer = chain.run({\"question\": \"What is photosynthesis?\"})\n print(answer)\n ```\n\n That's a tiny example; real apps might add a retriever, memory, or an agent on top of it.\n\n- **Why developers like it:** \n - **Modularity:** You can swap out the LLM, the vector store, or the prompt without rewriting everything. \n - **Productivity:** Lots of ready-made components (chains, agents, memory types) speed up prototyping. \n - **Community:** A growing ecosystem of tutorials, integrations, and open-source extensions.\n\n**Bottom line:** LangChain is a toolbox that turns raw language-model APIs into full-featured, context-aware applications, handling everything from prompt management to external tool integration. It lets you focus on *what* you want the app to do, while it takes care of *how* to wire the LLM together with other services.",
"additional_kwargs": {},
"response_metadata": {
"tokenUsage": {
"completionTokens": 776,
"promptTokens": 91,
"totalTokens": 867
},
"finish_reason": "stop",
"id": "chatcmpl-5344708c-0d53-4ed1-bdad-9db8b6b07052",
"object": "chat.completion",
"created": 1786987983,
"model": "openai/gpt-oss-120b",
"usage": {
"queue_time": 0.30476153,
"prompt_tokens": 91,
"prompt_time": 0.004791124,
"completion_tokens": 776,
"completion_time": 1.632367671,
"total_tokens": 867,
"total_time": 1.637158795,
"completion_tokens_details": {
Ln 18, Col 18 Spaces: 4 UTF-8 CRLF TypeScript

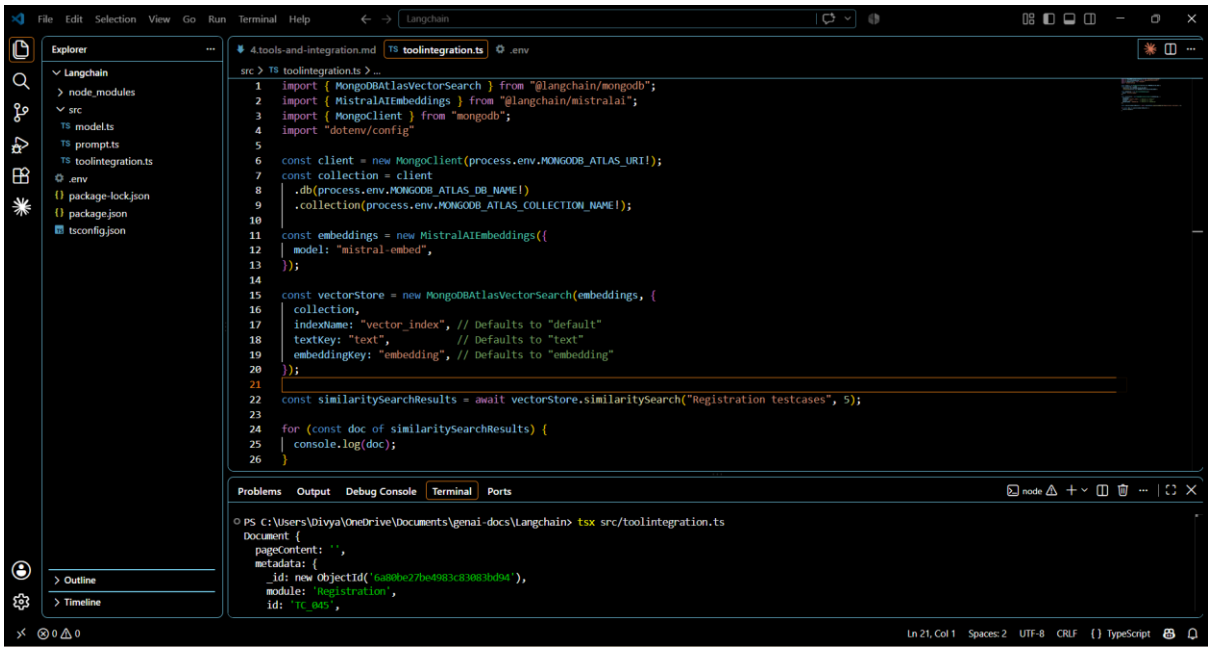

File Edit Selection View Go Run Terminal Help
Langchain
Explorer
Langchain
node_modules
src
model.ts
.env
package-lock.json
package.json
tsconfig.json
Outline
Timeline
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/model.ts
"object": "chat.completion",
"created": 1786987983,
"model": "openai/gpt-oss-120b",
"usage": {
"queue_time": 0.30476153,
"prompt_tokens": 91,
"prompt_time": 0.004791124,
"completion_tokens": 776,
"completion_time": 1.632367671,
"total_tokens": 867,
"total_time": 1.637158795,
"completion_tokens_details": {
"reasoning_tokens": 46
}
},
"usage_breakdown": null,
"system_fingerprint": "fp_4708260202",
"x_groq": {
"id": "req_01m08chah7eaa9srgah8fzq912",
"seed": 1403737283
},
"service_tier": "on_demand",
"model_provider": "groq"
},
"tool_calls": [],
"invalid_tool_calls": [],
"usage_metadata": {
"input_tokens": 91,
"output_tokens": 776,
"total_tokens": 867
}
}
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain>
Ln 18, Col 18 Spaces: 4 UTF-8 CRLF TypeScript

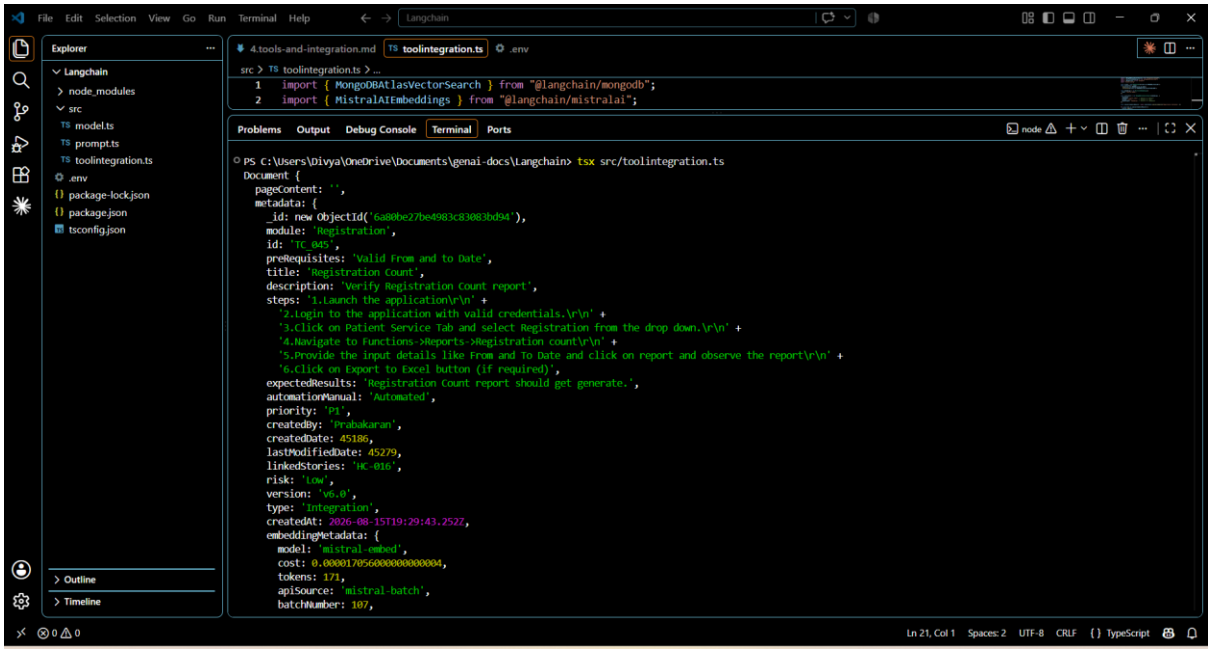
Prompt.ts


File Edit Selection View Go Run Terminal Help
Langchain
Explorer
Langchain
node_modules
src
model.ts
prompt.ts
.env
package-lock.json
package.json
tsconfig.json
tsconfig.json package.json model.ts 3.prompt-template.md prompt.ts .env
src > prompt.ts > ...
1 import { ChatPromptTemplate } from "@langchain/core/prompts";
2 import { SystemMessagePromptTemplate } from "@langchain/core/prompts";
3
4 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
5 const chatPrompt = ChatPromptTemplate.fromMessages([
6 ["system", "You are a helpful assistant."],
7 message,
8]);
9 const formattedChatPrompt = await chatPrompt.invoke({
10 text: "Explain about Langflow and Langchain in short",
11 });
12 console.log(formattedChatPrompt);
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/model.ts
}
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/prompt.ts
ChatPromptValue {
lc_serializable: true,
lc_kwargs: {
messages: [
SystemMessage {
"content": "You are a helpful assistant.",
"additional_kwargs": {},
"response_metadata": {}
},
SystemMessage {
"content": "Explain about Langflow and Langchain in short",
"additional_kwargs": {},
"response_metadata": {}
}
]
},
lc_namespace: ['langchain_core', 'prompt_values'],
messages: [
Outline
Timeline
Ln 12, Col 34 Spaces: 4 UTF-8 CRLF TypeScript

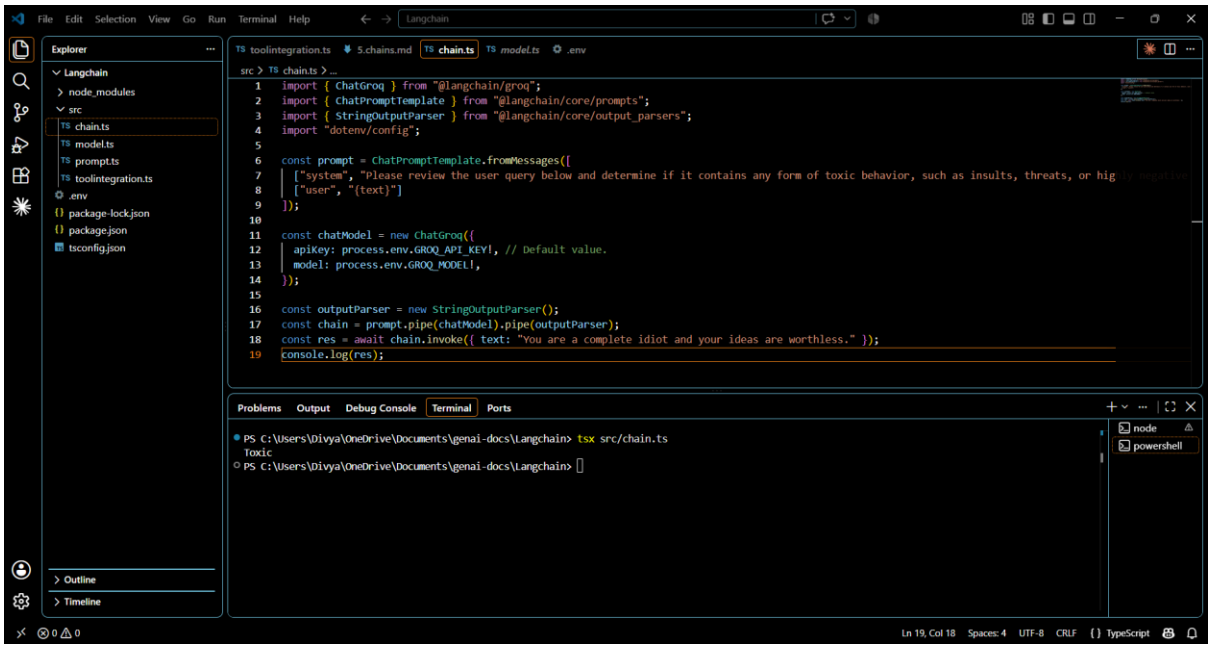

File Edit Selection View Go Run Terminal Help
Langchain
Explorer
Langchain
node_modules
src
model.ts
prompt.ts
.env
package-lock.json
package.json
tsconfig.json
tsconfig.json package.json model.ts 3.prompt-template.md prompt.ts .env
src > prompt.ts > ...
1 import { ChatPromptTemplate } from "@langchain/core/prompts";
2 import { SystemMessagePromptTemplate } from "@langchain/core/prompts";
3
4 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
5 const chatPrompt = ChatPromptTemplate.fromMessages([
6 ["system", "You are a helpful assistant."],
7 message,
8]);
9 const formattedChatPrompt = await chatPrompt.invoke({
10 text: "Explain about Langflow and Langchain in short",
11 });
12 console.log(formattedChatPrompt);
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/prompt.ts
]
},
lc_namespace: ['langchain_core', 'prompt_values'],
messages: [
SystemMessage {
"content": "You are a helpful assistant.",
"additional_kwargs": {},
"response_metadata": {}
},
SystemMessage {
"content": "Explain about Langflow and Langchain in short",
"additional_kwargs": {},
"response_metadata": {}
}
]
}
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain>
Outline
Timeline
Ln 12, Col 34 Spaces: 4 UTF-8 CRLF TypeScript

Toolintegration.ts





Chain.ts


File Edit Selection View Go Run Terminal Help
Langchain
Explorer
Langchain
node_modules
src
chain.ts
model.ts
prompt.ts
toolintegration.ts
.env
package-lock.json
package.json
tsconfig.json
toolintegration.ts
5.chains.md
chain.ts
model.ts
.env
src > chain.ts > ...
1 import { ChatGroq } from "@langchain/groq";
2 import { ChatPromptTemplate } from "@langchain/core/prompts";
3 import { StringOutputParser } from "@langchain/core/output_parsers";
4 import "dotenv/config";
5
6 const prompt = ChatPromptTemplate.fromMessages([
7 ["system", "Please review the user query below and determine if it contains any form of toxic behavior, such as insults, threats, or hig
8 ["user", "{text}"]
9]);
10
11 const chatModel = new ChatGroq({
12 apiKey: process.env.GROQ_API_KEY!, // Default value.
13 model: process.env.GROQ_MODEL!,
14 });
15
16 const outputParser = new StringOutputParser();
17 const chain = prompt.pipe(chatModel).pipe(outputParser);
18 const res = await chain.invoke({ text: "You are a complete idiot and your ideas are worthless." });
19 console.log(res);
Problems Output Debug Console Terminal Ports
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/chain.ts
Toxic
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain>
node
powershell
Outline
Timeline
Ln 19, Col 18 Spaces: 4 UTF-8 CRLF { } TypeScript